CareLoop

Shared Technical Contract v3.0

Python · FastAPI · Supabase PostgreSQL · Pydantic · LangChain + Groq

---

Architecture, database schema, API contract, event formats, state machine, error handling, ownership and integration rules for a 3-person, 20-hour build.

Purpose

This document is the proposed shared contract for all three developers. Person 1 must compare it with the existing repository and Supabase schema before applying migrations. Preserve working code and existing data; agree on deviations before implementation.

Core product principle

CareLoop tracks continuity across care actions and handoffs. A patient report is not clinical verification, and a completed action does not prove that its result reached the next responsible person.

1. Architecture

Recommended flow: coordinator dashboard, patient-message input and department simulator → FastAPI routes → Pydantic validation → workflow services and optional AI service → Supabase PostgreSQL → API response → refreshed dashboard.

Component	Owner	Contract
Database schema and migrations	Person 1	One agreed schema; no independent destructive migrations.
State machine and dependencies	Person 1	Only authority for care-action transitions.
FastAPI routes and event processing	Person 1	Validate every state-changing request.
Dashboard and API client	Person 2	Render backend state; never write directly to Supabase.
AI classification/extraction	Person 3	Return validated proposals; never directly modify workflow state.
Fixtures and end-to-end scenario	All 3	Use the same synthetic patient, action and event IDs.

Request-to-state flow

- Validate request structure with Pydantic.
- Validate actor, resource, authorization and allowed operation.
- Check event_id for duplicate or conflicting submissions.
- Validate the transition and prerequisites in the state-machine service.
- Persist the event and related state changes atomically.
- Write audit history and return the authoritative resulting state.
- Dashboard refreshes through the API.

Important: multiple independent Supabase Python client calls are not automatically atomic. Use a PostgreSQL transaction or database function/RPC for operations that update multiple related rows.

2. Canonical statuses

Care plans

Status	Meaning
DRAFT	Plan is being prepared.
PENDING_APPROVAL	Proposed plan awaits authorized review.
APPROVED	Plan approved.
SUPERSEDED	Replaced by a newer version.
CANCELLED	Cancelled through an authorized operation.

Care actions

Status	Meaning
PENDING	Exists, but prerequisites are not satisfied.
READY	Prerequisites satisfied; work can begin.
IN_PROGRESS	Work has started.
AWAITING_VERIFICATION	Completion reported but not verified.
VERIFIED_COMPLETE	Required evidence accepted by an authorized verifier.
BLOCKED	Explicit operational impediment prevents progress.
CANCELLED	Cancelled through an authorized operation.

OVERDUE is derived from due_at and the current state; it is not a care-action status.

Handoffs, exceptions and event processing

Entity	Allowed values / lifecycle
Handoff	PENDING → SENT → ACKNOWLEDGED; REJECTED or CANCELLED are non-success terminal outcomes.
Exception	OPEN → IN_PROGRESS → RESOLVED; authorized dismissal is DISMISSED.
Event processing	RECEIVED, PROCESSED, REJECTED.

Action completion and handoff acknowledgment are separate. A patient report does not independently produce VERIFIED_COMPLETE.

3. Database schema

Use UUID primary keys, foreign keys, UTC timestamps with TIMESTAMPTZ, database constraints and indexes. This is a target logical schema; adapt it to existing Supabase tables before migration.

patients

Column	Type / rules
id	UUID primary key
external_ref	TEXT, optional synthetic/HMS reference
display_name	TEXT, required
created_at	TIMESTAMPTZ, default current timestamp

care_plans

Column	Type / rules
id	UUID primary key
patient_id	UUID FK → patients.id
version	INTEGER; unique within patient plan lineage
status	Canonical plan status
source_text	TEXT, optional source plan text
created_by	TEXT actor identifier
approved_by	TEXT, nullable
approved_at	TIMESTAMPTZ, nullable
created_at / updated_at	TIMESTAMPTZ

Recommended constraint: UNIQUE(patient_id, version). AI extraction creates proposed actions, not an approved clinical plan.

care_actions

Column	Type / rules
id	UUID primary key
care_plan_id	UUID FK → care_plans.id
title / description	TEXT; title required, description optional
action_type	TEXT: DIAGNOSTIC, REFERRAL, FOLLOW_UP, REVIEW, etc.
status	Canonical action status
responsible_role	TEXT, e.g. DIAGNOSTICS or CARDIOLOGY
due_at	TIMESTAMPTZ, nullable
completion_reported_at	TIMESTAMPTZ, nullable
verified_at / verified_by	Nullable verification metadata
created_at / updated_at	TIMESTAMPTZ

Only an authorized verification operation may set verified_at and verified_by.

action_dependencies

Column	Type / rules
id	UUID primary key
action_id	UUID FK → dependent care_actions.id
depends_on_action_id	UUID FK → prerequisite care_actions.id
requirement_type	ACTION_VERIFIED or HANDOFF_ACKNOWLEDGED
required_handoff_id	Nullable FK → handoffs.id; required for HANDOFF_ACKNOWLEDGED
created_at	TIMESTAMPTZ

- UNIQUE(action_id, depends_on_action_id, requirement_type).
- action_id must differ from depends_on_action_id.
- Reject dependency cycles when approving a plan.
- Because required_handoff_id refers to handoffs, create both tables and add the circular FK in a migration that handles ordering correctly.

handoffs

Column	Type / rules
id	UUID primary key
action_id	UUID FK → care_actions.id
recipient_role / recipient_id	TEXT; role required, recipient ID optional
status	Canonical handoff status
sent_at / acknowledged_at	Nullable TIMESTAMPTZ
acknowledged_by	TEXT, nullable
deadline_at	TIMESTAMPTZ, required for monitored handoffs
rejection_reason	TEXT, nullable
created_at	TIMESTAMPTZ

action_evidence

Column	Type / rules
id	UUID primary key
action_id	UUID FK → care_actions.id
evidence_type	DEPARTMENT_CONFIRMATION, RESULT_REFERENCE, COORDINATOR_REVIEW, etc.
source / reference	TEXT; source required by policy, reference optional
submitted_by	TEXT actor identifier
verified_by / verified_at	Nullable verification metadata
created_at	TIMESTAMPTZ

Prefer evidence metadata or references to unnecessary clinical documents. Patient reports are not automatically accepted as verified evidence.

care_events

Column	Type / rules
id	UUID primary key
event_id	UUID unique external/idempotency ID
event_type	Canonical event type
action_id / handoff_id	Nullable FKs to affected resources
source	Canonical event source
actor_id / actor_role	Actor metadata; role required
occurred_at / received_at	TIMESTAMPTZ
processing_status	RECEIVED, PROCESSED or REJECTED
payload	JSONB validated event details
payload_hash	Optional; detects conflicting event-ID reuse
result_snapshot	Optional JSONB summary for safe replay
error_code	Nullable

Same event ID + same payload: do not repeat state changes; return the prior result when possible. Same event ID + different payload: reject with IDEMPOTENCY_CONFLICT.

exceptions

Column	Type / rules
id	UUID primary key
action_id / handoff_id	Nullable FKs
exception_type	HANDOFF_OVERDUE, BARRIER_REPORTED, etc.
status	Canonical exception status
assigned_role	TEXT
reason	TEXT required
resolution_note	TEXT nullable
created_at / resolved_at	TIMESTAMPTZ
resolved_by	TEXT nullable

Use partial unique indexes to prevent duplicate active exceptions for the same handoff, and separately for the same action and exception type when no handoff is involved. Active statuses: OPEN and IN_PROGRESS.

audit_log

Column	Type / rules
id	UUID primary key
event_id	Nullable FK → care_events.id
entity_type / entity_id	Entity type and affected UUID
operation	STATUS_CHANGED, ACKNOWLEDGED, RESOLVED, etc.
actor_id / actor_role	Actor metadata
before_state / after_state	Optional JSONB
created_at	TIMESTAMPTZ

Audit records should be append-only from the application's perspective. Resolving an exception must not delete its history.

Recommended indexes

- care_plans(patient_id, status)
- care_actions(care_plan_id, status) and care_actions(status, due_at)
- action_dependencies(action_id) and action_dependencies(depends_on_action_id)
- handoffs(status, deadline_at)
- exceptions(status, assigned_role, created_at)
- UNIQUE care_events(event_id)
- audit_log(entity_type, entity_id, created_at)

For the 20-hour build, prioritize plans, actions, dependencies, handoffs, events, exceptions and audit history. Add action_evidence if not already represented and verification needs it.

4. Event contract

Canonical event types

Event type	Meaning
ACTION_STARTED	Work on an action started.
COMPLETION_REPORTED	Completion reported, not necessarily verified.
COMPLETION_VERIFIED	Authorized verifier accepted required evidence.
HANDOFF_SENT	Information sent to recipient.
HANDOFF_ACKNOWLEDGED	Recipient confirmed receipt.
HANDOFF_REJECTED	Recipient rejected the handoff.
BARRIER_REPORTED	A barrier to action completion was reported.
RESCHEDULE_REQUESTED	Rescheduling was requested.

Canonical sources

PATIENT, DEPARTMENT, CARE_COORDINATOR, DEPARTMENT_SIMULATOR, SYSTEM. In a production system, derive or validate actor identity and permissions from authentication rather than trusting client-supplied roles.

Pydantic request model

```
# app/schemas/events.py
from datetime import datetime
from typing import Any, Literal
from uuid import UUID
from pydantic import BaseModel, ConfigDict, Field

EventType = Literal[
    "ACTION_STARTED", "COMPLETION_REPORTED", "COMPLETION_VERIFIED",
    "HANDOFF_SENT", "HANDOFF_ACKNOWLEDGED", "HANDOFF_REJECTED",
    "BARRIER_REPORTED", "RESCHEDULE_REQUESTED",
]
EventSource = Literal[
    "PATIENT", "DEPARTMENT", "CARE_COORDINATOR",
    "DEPARTMENT_SIMULATOR", "SYSTEM",
]

class Actor(BaseModel):
    id: str | None = None
    role: str

class CareEventCreate(BaseModel):
    model_config = ConfigDict(extra="forbid")
    event_id: UUID
    event_type: EventType
    action_id: UUID | None = None
    handoff_id: UUID | None = None
    source: EventSource
    actor: Actor
    occurred_at: datetime
    payload: dict[str, Any] = Field(default_factory=dict)
```

Example event

```
{
  "event_id": "e25b4dc0-5fc7-4a3e-9c39-69e9acbe0301",
  "event_type": "HANDOFF_ACKNOWLEDGED",
  "action_id": "aaaaaaaa-aaaa-4aaa-8aaa-aaaaaaaaaaaa",
  "handoff_id": "bbbbbbbb-bbbb-4bbb-8bbb-bbbbbbbbbbbb",
  "source": "DEPARTMENT_SIMULATOR",
  "actor": {"id": "demo-cardiology", "role": "CARDIOLOGY"},
  "occurred_at": "2026-10-13T08:00:00Z",
  "payload": {"receipt_reference": "DEMO-RESULT-001"}
}
```

Processing rules

- Validate schema, source/event combination, resource existence and authorization.
- Check event_id for duplicates/conflicting payloads.
- Check current state and allowed transition.
- Apply state changes, dependency updates and audit writes atomically.
- Return the resulting authoritative state and affected exceptions.
- Never allow a generic client-supplied status update to bypass the state machine.

5. API contract

Base URL: http://localhost:8000 | Prefix: /api | Development docs: /docs

Standard response envelopes

```
{
  "success": true,
  "data": {},
  "meta": {"request_id": "req-example-001"},
  "error": null
}

{
  "success": false,
  "data": null,
  "meta": {"request_id": "req-example-001"},
  "error": {
    "code": "INVALID_STATE_TRANSITION",
    "message": "The requested transition is not allowed.",
    "details": {}
  }
}
```

Endpoint inventory

Method	Endpoint	Owner / purpose
GET	/api/health	P1: health check
GET	/api/care-plans/{plan_id}	P1: plan and actions
POST	/api/care-plans	P1: create draft
POST	/api/care-plans/{plan_id}/parse	P1 + P3: extract proposals
POST	/api/care-plans/{plan_id}/approve	P1: approve reviewed plan
GET	/api/patients/{patient_id}/care-journey	P1: journey, actions, dependencies
GET	/api/care-actions/{action_id}	P1: action details
POST	/api/care-actions/{action_id}/start	P1: start eligible action
POST	/api/care-actions/{action_id}/report-completion	P1: report, not verify
POST	/api/care-actions/{action_id}/verify-completion	P1: authorized verification
POST	/api/handoffs	P1: create handoff
POST	/api/handoffs/{handoff_id}/send	P1: record sending
POST	/api/handoffs/{handoff_id}/acknowledge	P1: record receipt
POST	/api/care-events	P1: process canonical event
GET	/api/exceptions	P1: list/filter exceptions
PATCH	/api/exceptions/{exception_id}	P1: assign/progress/resolve/dismiss
GET	/api/audit	P1: entity history
POST	/api/ai/classify-message	P3: classify patient message
POST	/api/ai/extract-care-plan	P3: optional extraction

Care journey response shape

```
{
  "patient": {"id": "patient-uuid", "display_name": "Demo Patient"},
  "care_plan": {"id": "plan-uuid", "status": "APPROVED", "version": 1},
  "actions": [
    {
      "id": "ecg-action-uuid",
      "title": "ECG",
      "status": "VERIFIED_COMPLETE",
      "responsible_role": "DIAGNOSTICS",
      "dependencies": []
    },
    {
      "id": "review-action-uuid",
      "title": "Cardiology review",
      "status": "PENDING",
      "responsible_role": "CARDIOLOGY",
      "dependencies": [{
        "requirement_type": "HANDOFF_ACKNOWLEDGED",
        "satisfied": false
      }]
    }
  ],
  "open_exception_count": 1
}
```

If multiple plan versions exist, document which version the journey endpoint returns. The response must be generated from persisted backend data, not hardcoded UI data.

POST /api/care-events

Request: the canonical event envelope in Section 4.

```
{
  "event_id": "e25b4dc0-5fc7-4a3e-9c39-69e9acbe0301",
  "processing_status": "PROCESSED",
  "action": {"id": "review-action-uuid", "status": "READY"},
  "affected_exception_ids": []
}
```

READY is illustrative only. The backend computes the actual result from current state, event validity and prerequisites.

POST /api/handoffs

```
{
  "action_id": "aaaaaaaa-aaaa-4aaa-8aaa-aaaaaaaaaaaa",
  "recipient_role": "CARDIOLOGY",
  "recipient_id": "demo-cardiology",
  "deadline_at": "2026-10-13T08:30:00Z"
}
```

Creating a handoff does not imply it was sent or acknowledged.

POST /api/handoffs/{handoff_id}/acknowledge

```
{
  "event_id": "c25b4dc0-5fc7-4a3e-9c39-69e9acbe0302",
  "actor": {"id": "demo-cardiology", "role": "CARDIOLOGY"},
  "occurred_at": "2026-10-13T08:15:00Z",
  "payload": {"receipt_reference": "DEMO-RESULT-001"}
}
```

The server validates recipient/actor, records acknowledgment and audit history, resolves the applicable exception and reevaluates dependent actions.

GET /api/exceptions

Supported query parameters: status, assigned_role, limit (default 20, capped) and cursor if pagination is implemented.

```
{
  "items": [{
    "id": "exception-uuid",
    "exception_type": "HANDOFF_OVERDUE",
    "status": "OPEN",
```

```
    "assigned_role": "CARDIOLOGY",
    "reason": "Required result acknowledgment has not arrived.",
    "action_id": "ecg-action-uuid",
    "handoff_id": "handoff-uuid",
    "created_at": "2026-10-13T08:30:00Z"
  }],
  "next_cursor": null
}
```

PATCH /api/exceptions/{exception_id}

```
{
  "status": "IN_PROGRESS",
  "assigned_role": "CARDIOLOGY",
  "resolution_note": null
}
```

Only documented transitions are allowed. Resolution or dismissal requires an authorized actor and a reason. Resolving an exception does not verify the underlying action.

Report completion vs verify completion

```
POST /api/care-actions/{action_id}/report-completion
{
  "event_id": "d25b4dc0-5fc7-4a3e-9c39-69e9acbe0303",
  "source": "PATIENT",
  "actor": {"id": "demo-patient", "role": "PATIENT"},
  "occurred_at": "2026-10-13T08:20:00Z",
  "payload": {"message": "I completed the test."}
}
```

Expected result: AWAITING_VERIFICATION when the transition is valid.

```
POST /api/care-actions/{action_id}/verify-completion
{
  "event_id": "e25b4dc0-5fc7-4a3e-9c39-69e9acbe0304",
  "evidence_id": "evidence-uuid",
  "verification_note": "Department confirmation checked."
}
```

Expected result: VERIFIED_COMPLETE only with valid evidence and an authorized verifier. Define the prototype's trusted-actor simulation explicitly.

POST /api/ai/classify-message

```
Request:
{
  "message": "I could not attend the test because I had no transport.",
  "action_id": "ecg-action-uuid"
}

Response data:
{
  "intent": "BARRIER_REPORTED",
  "action_id": "ecg-action-uuid",
  "confidence": 0.92,
  "requires_human_review": true
}
```

Illustrative confidence only. Supported intents: COMPLETION_REPORTED, BARRIER_REPORTED, RESCHEDULE_REQUESTED, QUESTION, URGENT_CONCERN, UNCLEAR. AI proposes an intent; backend validates and decides whether to process an event.

Plan parsing and approval

```
POST /api/care-plans/{plan_id}/parse
{
  "plan_id": "plan-uuid",
  "proposed_actions": [
    {
      "title": "ECG",
      "action_type": "DIAGNOSTIC",
      "responsible_role": "DIAGNOSTICS",
      "depends_on_titles": []
    },
    {
      "title": "Cardiology review",
      "action_type": "REVIEW",
      "responsible_role": "CARDIOLOGY",
      "depends_on_titles": ["ECG"]
    }
  ],
  "requires_human_review": true
}
```

Parsing returns proposals only. The approval endpoint validates dependencies, rejects cycles and persists the approved plan/actions consistently. This path is optional if starting from an existing synthetic approved plan.

GET /api/audit

Query by entity_type and entity_id. Return ordered audit entries containing operation, actor, timestamp and before/after state where available. Clients cannot alter or delete audit history.

6. State machine and dependency rules

Current state	Event / condition	Result
PENDING	All prerequisites satisfied	READY
READY	ACTION_STARTED	IN_PROGRESS
IN_PROGRESS	COMPLETION_REPORTED	AWAITING_VERIFICATION
AWAITING_VERIFICATION	Valid evidence accepted by authorized verifier	VERIFIED_COMPLETE
READY or IN_PROGRESS	Authorized barrier blocks progress	BLOCKED
BLOCKED	Barrier resolved and prerequisites satisfied	READY
Eligible non-terminal action	Authorized cancellation	CANCELLED

- ACTION_VERIFIED dependency is satisfied only when the prerequisite is VERIFIED_COMPLETE.
- HANDOFF_ACKNOWLEDGED dependency is satisfied only when the referenced handoff is ACKNOWLEDGED.
- A PENDING action becomes READY only when all dependencies are satisfied.
- Reject dependency cycles at plan approval.
- PENDING may represent waiting on prerequisites; reserve BLOCKED for an explicit impediment.
- Reopening verified actions requires a separate authorized correction workflow.

Handoff monitoring

- A handoff is overdue when it has been sent, its deadline has passed, and it has not been acknowledged, rejected or cancelled.
- For the hackathon, a deterministic monitor/demo trigger is acceptable; do not depend on waiting a real hour.
- Create one active HANDOFF_OVERDUE exception per handoff.
- On acknowledgment: persist event, update handoff, resolve associated exception, write audit, reevaluate dependencies and return updated state.
- Acknowledgment does not prove the underlying clinical action was completed.

7. Error codes

HTTP	Code	Use
400	VALIDATION_ERROR	Malformed request, if configured as 400
401	UNAUTHENTICATED	Authentication required
403	FORBIDDEN	Actor not authorized
404	RESOURCE_NOT_FOUND	Resource does not exist
409	INVALID_STATE_TRANSITION	Operation disallowed from current state
409	IDEMPOTENCY_CONFLICT	Event ID reused with different payload
409	RESOURCE_CONFLICT	Conflicting plan version or active exception
422	DEPENDENCY_NOT_SATISFIED	Prerequisites unmet
422	EVIDENCE_REQUIRED	Verification lacks required evidence
422	EVIDENCE_INVALID	Evidence rejected
429	RATE_LIMITED	Rate limit exceeded, if implemented
503	AI_UNAVAILABLE	AI feature unavailable; core workflow remains usable
500	INTERNAL_ERROR	Unexpected server error

FastAPI normally uses 422 for request-model validation. If the team wants 400, configure a handler explicitly; otherwise document and consistently use 422.

8. Repository structure and ownership

```
care-loop/
■■■ app/
■   ■■■ main.py
■   ■■■ api/
■   ■■■ schemas/
■   ■■■ services/
■   ■■■ db/
■   ■■■ ai/
■■■ migrations/
■■■ tests/
■■■ docs/CONTRACT.md
■■■ requirements.txt
■■■ .env.example
```

Files / areas	Owner
app/main.py, app/db/, migrations	Person 1
app/api/, app/services/	Person 1
app/schemas/events.py and shared response models	Person 1, reviewed by all
Frontend components and API client	Person 2
app/ai/, AI-specific tests and fixtures	Person 3
docs/CONTRACT.md	Person 1 maintains; all agree to changes
Integration tests and demo fixture IDs	All 3

If the repository already has a different Python package layout, preserve it and keep the same ownership boundaries.

9. Integration checkpoints — 20-hour plan

Time	Checkpoint / merge gate
H0–1	Contract frozen; Person 1 inspects schema and routes; agree on statuses, events, endpoints and shared IDs.
H1–3	FastAPI skeleton and DB connection; dashboard mock; AI schema and fixtures.
H3–5	Merge contract/backend skeleton; verify startup, seeded reads, mock UI and schema-valid AI output.
H5–8	Implement state machine/dependencies; dashboard exceptions; one AI feature.
H8–10	Merge working modules; real API data reaches UI; event submission persists a state change.
H10–13	Finish handoff monitor, exception resolution, audit and dashboard acknowledgment flow.
H13–15	Full end-to-end demo works: missing handoff → exception → acknowledgment → resolution → downstream eligibility.
H15–17	Regression and polish; feature freeze at H17.
H17–19	Run demo from clean startup twice; prepare fixtures/fallback.
H19–20	Final acceptance; document startup; promote tested develop commit to main.

Git policy

- Use feature/backend-core, feature/coordinator-dashboard and feature/ai-workflows from develop; keep main stable.
- Push small working commits; merge only tested branches at checkpoints.
- Person 1 owns shared files, database migrations and the final integration.
- After merges, rebase or merge the updated develop into each feature branch and rerun tests.
- At H17 freeze features. Promote to main only after end-to-end acceptance.

10. Acceptance test checklist

Area	Acceptance criteria
Core workflow	Seeded plan and patient load; unmet dependencies prevent READY; valid event persists.
Verification	Patient report does not become VERIFIED_COMPLETE by itself.
Handoff	Missing acknowledgment creates exception; dashboard shows responsible role; valid acknowledgment resolves exception and records audit.
Dependencies	Downstream action becomes READY only when every prerequisite is satisfied.
Reliability	Duplicate event ID does not repeat transition; invalid transition returns useful error without partial corruption.
AI	Malformed output or model outage does not stop core workflow.
Demo	Works with seeded synthetic data and no real messaging provider.

Scope-cutting rules

If delayed	Simplify	Preserve
Migrations	Use existing schema, add only essential fields	Reliable state and event history
State machine	Implement minimum allowed transitions	Backend remains state authority
Frontend	One journey page and exception panel	Visible gap and resolution
LangChain/Groq	Use fixed fixtures/fallback	Core workflow works without AI
Deadline monitor	Use deterministic demo trigger	Exception lifecycle and audit
Messaging	Simulate acknowledgment	Persisted backend event
Styling	Simple consistent dashboard	Readable statuses and ownership

11. Shared team agreement

- Person 1 verifies existing Supabase schema before migrations.
- All modules use exact canonical status and event values.
- Backend state machine is the only authority for care-action transitions.
- Frontend uses the API, never direct Supabase writes.
- AI returns validated proposals and cannot independently verify completion.
- Every state-changing event has a unique ID and defined duplicate behavior.
- Handoff acknowledgment and action completion are separate.
- Multi-row transitions and audit writes are atomic where required.
- All three developers use the same synthetic patient, plan, action and handoff IDs.
- The full demo runs with Groq unavailable.

First action

Person 1 creates docs/CONTRACT.md, compares the proposed schema against the actual Supabase tables, and shares the agreed version with Persons 2 and 3. Do not implement against an unverified schema.